

紙芝居アプリ ソフトウェア開発者向け資料

この資料は、TMPose 紙芝居を構成するアプリと、開発に関連するライブラリをソフトウェア開発者向けにまとめたものです。

対象アプリ/DSL: `kamishibai=3.1`

1. アプリ

1.1 「参加型」AI紙芝居アプリ

「参加型」AI紙芝居アプリは、オープンソースライセンス(MPL2.0)で公開しています。

3.1 では、Asset Manager によるプロジェクト内/外部アセットとテキストアセットの統合、Temporary Variables による状態管理、Runtime Expression による条件評価、Async Input による入力待ち、TMPose によるポーズ認識を組み合わせで DSL を実行します。

- tmpose-kamishibai: アプリ本体・ドキュメント・汎用ビルドツール
 - <https://github.com/kubohiroya/tmpose-kamishibai>
- tmpose-kamishibai-samples: 公開用台本・サンプル固有アセット
 - <https://kubohiroya.github.io/tmpose-kamishibai-samples/>
 - <https://github.com/kubohiroya/tmpose-kamishibai-samples>

1.2 `kamishibai.sb3` の依存物管理方針

配布する `kamishibai.sb3` は、台本固有の画像・音声アセットを組み込まない汎用実行環境として、Git 管理された展開ソースから生成します。機能拡張とアセットは、提供元と用途に応じて次のように扱います。

- Animated Text や Temporary Variables など、`extensions.turbowarp.org` で提供される TurboWarp Extension Gallery 採用済みの機能拡張は、外部 URL を参照します。本資料では、これらを TurboWarp 標準の機能拡張と呼びます。
- Asset Manager、TMPose、Text Lines、Runtime Expression、Async Input など、tmpose-kamishibai 固有の非サンドボックス機能拡張は、JavaScript を base64 データ URL に変換して `kamishibai.sb3` 内へ格納します。
- 台本で使用する画像・音声アセットは `kamishibai.sb3` に組み込まず、台本ファイルの `asset=` 行から外部 URL を参照します。

- TurboWarp 標準の機能拡張と外部アセットは、提供元が内容と URL を安定して維持することを前提に利用します。SHA-3 などのハッシュ値による独自の完全性検証は行いません。

外部アセットの提供元には、HTTPS で取得できること、ブラウザからの取得に必要な CORS 設定が行われていること、URL が長期的に維持されることを求めます。配信内容または URL が変更された場合は、必要に応じて台本ファイルを更新します。

1.3 正本と生成物

`app/` ディレクトリを紙芝居アプリの正本として Git 管理します。ルートの `kamishibai.sb3` や `site/downloads/` 配下の SB3 バイナリは正本として管理しません。

浦島太郎の正本は、本体とは別の `kubohiroya/tmpose-kamishibai-samples` リポジトリの `stories/urashima/` 以下で管理します。`source.txt`、画像・音声、アセットロック、生成設定を正本とし、公開ビルドで編集用 `_urashima.sb3`、再生用 `urashima.sb3`、変換済み `urashima.txt`、Web 版を生成します。生成物は浦島太郎の公開ページで MPL-2.0 により配信します。本体リポジトリでは、浦島太郎固有の SB3、台本、画像、音声を管理・配布しません。

`app/` の主な内容は次のとおりです。

- `project.source.json` : 整形済みの Scratch プロジェクト JSON
- `embedded-extensions.json` : 埋め込み拡張の ID、ファイル、データ URL 符号化方式
- `extensions/` : 個別ファイルへ復号したカスタム機能拡張
- `assets/` : 汎用実行環境自体が参照する画像・音声
- `sb3-source.json` : ZIP エントリの順序を含む展開ソースのマニフェスト

`app/project.source.json` の台本解析・実行用リストは空の初期状態で管理し、浦島太郎固有のターゲット、背景、コスチューム、音声は含めません。この不変条件は配布テストで検査します。

SB3 は用途ごとに次の場所へ生成します。どちらも Git 管理対象ではありません。

用途	生成先	生成コマンド
TurboWarp での再編集、ローカルテスト	<code>tmp/kamishibai.sb3</code>	<code>pnpm sb3:build</code>
GitHub Pages での配布	<code>dist/downloads/kamishibai.sb3</code>	<code>pnpm run build</code>

生成処理は ZIP エントリ順とタイムスタンプを固定します。同じ `app/` から生成した SB3 は bit-for-bit で一致します。`pnpm sb3:check` は、アセット参照と MD5、マニフェストの不足・余剰、埋め込み拡張の対応関係を検証します。

1.4 TurboWarpで編集した内容を取り込む

通常の更新手順は次のとおりです。

1. `app/` に未コミット差分がないことを確認する。
2. `pnpm sb3:build` を実行する。
3. `tmp/kamishibai.sb3` を TurboWarp で開き、編集する。
4. TurboWarp から編集済み SB3 を明示した場所へ保存する。
5. `pnpm sb3:import -- /path/to/edited-kamishibai.sb3` を実行する。
6. `git diff -- app` で、ブロック、拡張、アセットの差分を確認する。
7. 次の検証を順に実行する。

```
pnpm sb3:check
pnpm test
pnpm run build
```

`import` は、入力 SB3 から作成した候補と既存の `app/` を比較します。内容が同一なら書き換えません。相違がある場合、Git 管理外の出力や未コミット差分を既定では置換しません。Git 管理済みで clean な差分の確認だけを省略するときは `--yes`、未コミット差分も意図的に破棄するときだけ `--discard-local-changes` を追加します。後者は別指定であり、`--yes` だけでは未コミット差分を破棄できません。

カスタム機能拡張だけを修正する場合は、`app/extensions/` の JavaScript を直接編集できます。手作業でもコーディングエージェントでも、編集後は同じ3つの検証コマンドを実行します。TurboWarp GUI で行った変更とソースファイルへの部分修正を同時に取り込む場合は、先に `import` を完了して Git 差分を確認してから、部分修正を重ねます。

1.5 配布確認とロールバック

`pnpm run build` はサイト一式を作成し、`dist/downloads/kamishibai.sb3` が `app/` からの決定的生成結果と一致すること、ダウンロードページから参照されていること、ZIP 形式とプロジェクト構造が妥当であることを検証します。

リリースまたはマージ前には、生成 SB3 を TurboWarp で開き、少なくとも次を手動確認します。

- 読込エラーが表示されない。
- 緑の旗で表紙とメニューが表示される。
- 台本ファイルを読み込める。
- スペースキー、右矢印キー、下矢印キーの進行操作が機能する。

ソース更新を戻す場合は、該当コミットを `git revert` してから `pnpm sb3:build` と `pnpm run build` を再実行します。未コミットの `app/` を戻す操作は差分を失うため、先に `git diff -- app` を確認し、必要ならコミットまたは stash します。

importまたはbuildの途中で `.app.rollback-*` や `.kamishibai.sb3.rollback-*` が残った場合は、自動削除や再実行をせず、中身と元の出力を比較します。復旧対象を確定した後で元の場所へ戻し、通常の検証を実行します。配布に問題が見つかった場合は、GitHub Pagesを直前の検証済みコミットから再構築できます。SB3バイナリを正本としてリポジトリへ戻す必要はありません。

Loading表示とアセット読込順

汎用SB3は、アセット読込専用の組み込みスプライト `Loading` と組み込みコスチューム `loading` を持ちます。従来名称 `Hatching`（旧SB3データ上は `Hatchling`）は使用しません。

台本の `setLoadingCostume=名前1,名前2,...` は、`asset=` で登録する画像アセット名のリストです。ランタイムはストーリー開始時に前回の設定を空へ戻し、scene 0の全コマンドを解析した後で次の順序を確定します。このため、`setLoadingCostume` と対象 `asset=` の記述順には依存しません。

1. 指定名の前後空白を除去し、空要素と重複名を除外する。
2. 指定名がすべて `assetList` に存在することを検証する。未定義名があれば読込を停止する。
3. 指定されたLoading用アセットを、互いの記述順を保ったまま `assetList` の先頭へ移動する。
4. Loading用アセットをすべて登録してから、残りの通常アセットを登録する。
5. 通常アセットの読込開始前と完了後に、Loading用アセットを除いた `完了数 / 総数` を通知する。
6. 通常アセットの1始まりの読込番号をLoading用画像数で循環させ、`Loading` スプライトへ適用する。
7. 全アセット完了後に吹き出しと `Loading` スプライトを非表示にする。

Asset Managerの内部ブロック `setLoadingCostumes`、`prepareLoadingAssets`、`loadingAssetCount`、`loadingCostumeAt` が、設定の正規化、安定した優先順、除外件数、循環選択を担当します。これらはSB3内のランタイム実装用で、台本作者が直接呼び出すブロックではありません。`setLoadingCostume` がない場合、循環対象名は空となり、組み込みコスチュームをそのまま表示します。

進捗の分母は `assetList`の長さ - Loading用アセット数、分子は `現在のassetList番号 - Loading用アセット数` です。Loading用アセットだけの台本では通常アセットの進捗表示を開始せず、そのまま完了通知へ進みます。

1.6 汎用SB3・台本変換ビルダー

`@kubohiroya/tmpose-kamishibai` は、ベースSB3、外部参照付き台本、アセットロックマニフェストから、次の3成果物を同時に生成します。

```
<出力名>.sb3
<出力名>.txt
<出力名>.manifest.json
```

ビルダーはステージ背景、スプライトのコスチューム、ステージ音、スプライト音を追加します。ベース SB3 にあるブロック、拡張機能、変数、モニター、既存アセットは保持し、入力 SB3 と入力台本は書き換えません。台本は行単位で解析し、有効な `asset=` コマンドだけを変換します。コメント、他のコマンド、本文、行順、改行コードは保持します。

固定バージョンでの導入

消費側は浮動 `main` へ依存せず、検証済みの Git タグを `package.json` へ固定します。現在のリポジトリ間依存では npm レジストリ公開を前提にしません。

```
{
  "dependencies": {
    "@kubohiroya/tmpos-kamishibai": "github:kubohiroya/tmpos-kamishibai#v3.1.0"
  }
}
```

pnpm 11 は Git 依存パッケージの準備スクリプトを既定で拒否します。消費側の `pnpm-workspace.yaml` で、このパッケージだけを信頼対象として明示します。全依存のスクリプトを一括許可してはいけません。

```
allowBuilds:
  '@kubohiroya/tmpos-kamishibai': true
```

初回は `pnpm install` を実行し、生成された `pnpm-lock.yaml` を `package.json`、`pnpm-workspace.yaml` とともにコミットします。lockfile は Git タグを具体的なコミット SHA と取得元 tarball へ解決します。CI では `pnpm install --frozen-lockfile` を使用し、lockfile にない依存更新を拒否します。

リリースでは `package.json` のバージョン `3.1.0` と Git タグ `v3.1.0` を対応させ、公開後のタグを移動・削除しません。公開前のパッケージ内容は次のコマンドで tarball 化して確認できます。

```
pnpm pack
```

npm レジストリ公開は、リポジトリ間の固定依存を成立させるための必須条件ではありません。不特定の外部利用者へ標準的な SemVer パッケージとして提供する場合は、別のリリース Issue で公開と provenance を管理します。

JavaScript API

`package.json` の `./builder export` から `buildSb3Bundle` を読み込みます。

```
import {buildSb3Bundle} from '@kubohiroya/tmpose-kamishibai/builder';

const result = await buildSb3Bundle({
  baseSb3: 'kamishibai.sb3',
  sourceScript: 'source.txt',
  assetManifest: 'assets.lock.json',
  outputDirectory: 'dist',
  outputName: 'sample',
  profile: 'editor',
});

console.log(result.outputPaths);
```

`profile` は `editor` または `player` を必ず明示します。`editor` は変換済み台本を外部TXTだけに保持し、`player` は同じバイト列を外部TXTとSB3内の予約変数 `tmposeEmbeddedScript` の両方へ保持します。`baseSb3` と `sourceScript` にはファイルパスまたは `file:` URLを指定します。`assetManifest` にはファイルパス、`file:` URL、または同じ構造のJavaScriptオブジェクトを指定できます。オブジェクトを渡し、相対 `file:` 参照を使う場合は `manifestBaseDirectory` で基準ディレクトリを明示します。

組み込み台本は既定で1 MiBまでです。APIの `maxEmbeddedScriptBytes`、CLIの `--max-script-bytes` で1以上のバイト数へ変更できます。空台本、不正なUTF-8、上限超過、予約変数の欠損・重複・型不一致、または台本が入ったベースSB3は、既存成果物を更新せずエラーにします。

CLI

CLIはAPIと同じ `buildSb3Bundle` を呼び出し、生成仕様を重複実装しません。`--output` には拡張子を付けない出力ベース名を指定します。

```
tmpose-kamishibai build-sb3 \
  --base kamishibai.sb3 \
  --script source.txt \
  --assets assets.lock.json \
  --output dist/sample \
  --profile editor
```

追加オプションは次のとおりです。

オプション	意味
<code>--profile PROFILE</code>	<code>editor</code> または <code>player</code> を必ず指定する
<code>--allow-file-root DIR</code>	<code>file:</code> の許可ルートを追加する。複数回指定可能
<code>--allow-http</code>	平文HTTPを明示的に許可する
<code>--timeout-ms N</code>	1 リクエストのタイムアウト
<code>--max-asset-bytes N</code>	1 アセットの最大サイズ
<code>--max-script-bytes N</code>	組み込み台本の最大サイズ
<code>--max-redirects N</code>	HTTPリダイレクトの上限

入カマニフェスト

`assets.lock.json` は `formatVersion: 1` と 1 件以上の `assets` を持ちます。各アセットは、DSL 名、入力 URI、組み込み種別、`target`、SB3 内の名前、Content-Type、データ形式、サイズ、SHA-256、ライセンス参照、Scratch メタデータを固定します。

```

{
  "formatVersion": 1,
  "assets": [
    {
      "name": "forest",
      "uri": "file:assets/forest.svg",
      "kind": "backdrop",
      "target": "@stage",
      "sb3Name": "森",
      "contentType": "image/svg+xml",
      "dataFormat": "svg",
      "size": 1234,
      "sha256": "<64文字の16進数>",
      "license": "CC-BY-4.0: https://example.com/license",
      "metadata": {
        "bitmapResolution": 1,
        "rotationCenterX": 240,
        "rotationCenterY": 180
      }
    },
    {
      "name": "opening",
      "uri": "https://example.com/opening.wav",
      "kind": "stageSound",
      "target": "@stage",
      "sb3Name": "オープニング",
      "contentType": "audio/wav",
      "dataFormat": "wav",
      "size": 5678,
      "sha256": "<64文字の16進数>",
      "license": "CC0-1.0",
      "metadata": {
        "format": "",
        "rate": 48000,
        "sampleCount": 2800
      }
    }
  ]
}

```

kind と target の組み合わせは次のとおりです。

kind	target	変換後の台本参照
backdrop	@stage	backdrop:<sb3Name>
costume	スプライト名	costume:<target>:<sb3Name>
stageSound	@stage	sound:@stage:<sb3Name>
spriteSound	スプライト名	sound:<target>:<sb3Name>

DSL名は重複できません。同じtarget、同じコレクション、同じSB3名へ複数アセットを割り当てることもできません。既存SB3に同名アセットがある場合も、推測や上書きを行わずエラーにします。

セキュリティ、再現性、検証

`file:` 参照は既定でマニフェストのあるディレクトリ以下だけを許可します。`..` やシンボリックリンクの解決後に許可ルートを出るパスは拒否します。追加ルートはAPIの `allowedFileRoots` またはCLIの `--allow-file-root` で明示します。

HTTP(S)取得はステータス、Content-Type、実サイズ、ロック済みサイズ、SHA-256、タイムアウト、最大サイズ、リダイレクト回数を検証します。既定ではHTTPSだけを許可し、HTTPはAPIの `allowHttp: true` またはCLIの `--allow-http` を指定した場合だけ使用できます。

生成SB3はZIPエントリを `project.json` 先頭、残りをファイル名順に並べ、日時を1980年1月1日、圧縮レベルを固定します。`project.json` と出力manifestも正規化して書き出します。同じ入力と設定から生成したSB3、台本、manifestはbit-for-bitで一致します。

出力manifestの `formatVersion` は2です。使用パッケージとバージョン、`profile`、入力3点のSHA-256、各アセットの入力ロックとSB3ファイル名・アセットID・台本参照、出力SB3と台本のSHA-256を記録します。`script` には `mode` (`external` または `embedded`)、予約変数ID、UTF-8バイト数、SHA-256を記録します。出力確定前に、SB3内のtargetとアセットファイル、変換済み台本、manifestの全対応を照合し、`player` ではSB3内の台本、外部TXT、manifestのサイズとハッシュが一致することを再読込して確認します。

エラーとロールバック

アセット固有のエラーには処理段階、DSLアセット名、入力URIを含めます。欠損、HTTPエラー、Content-Type・サイズ・SHA-256不一致、target欠損、名前競合、危険なパスのいずれでも、検証が完了するまで既存出力を変更しません。

3成果物は同じ出力ディレクトリ内の一時領域で生成・検証します。確定時は既存3成果物をロールバック領域へ移してから新成果物へ置き換え、途中で失敗した場合は新成果物を除去して旧成果物を復元します。＜出力名＞.rollback-* が残っている場合は、前回処理が中断した可能性があるため自動上書きせず停止します。内容を確認して旧成果物を復元した後に再実行します。

パッケージに問題がある場合、消費側はGitタグ指定とlockfileを直前の検証済み版へ戻します。このビルダーは通常の本体ビルドへ直接組み込まれていないため、本体配布処理を変更せず切り戻せます。

1.7 成果物プロファイル

紙芝居アプリのSB3は、台本と物語固有アセットをどこに保持するかにより、generic、editor、playerの3プロファイルに分けます。プロファイルは用途と生成契約を表すものであり、SB3をTurboWarp Editorで編集できるかどうかを制限するアクセス権ではありません。そのため、playerは「再生専用」ではなく「再生用」と表記します。

プロファイル	ファイル名の例	台本	物語固有アセット	主な用途
generic	kamishibai.sb3	非埋め込み	非埋め込み	汎用の物語作成・再生用雛形
editor	_urashima.sb3	非埋め込み	埋め込み	物語作成者による台本編集・動作確認
player	urashima.sb3	埋め込み	埋め込み	配布・再生、PackagerによるWeb版生成

generic：汎用雛形

genericは本体リポジトリがapp/から生成するkamishibai.sb3です。公開する物語の台本、背景、キャラクター画像、効果音、BGMを含めません。利用者または物語作成者が台本を開き、台本のasset=定義に従ってアセットを読み込む従来の作成・再生フローを提供します。

genericはサンプル変換ビルダーの出力ではなく、editorとplayerを生成するためのベースアプリでもあります。本体リポジトリでは、特定の物語を追加せず、この汎用性を配布テストで維持します。

editor：物語作成者向け

editorは特定の物語で使用する画像・音声をSB3へ組み込みますが、台本は組み込みません。浦島太郎では_urashima.sb3を使用し、物語作成者が外部のurashima.txtを開いて編集・再読込しながら動作を確認します。台本を変更してもアセットを再取得する必要がないため、台本の反復編集とデバッグに使用できます。

ファイル名先頭の_は、物語作成者が内部的に使用する編集用成果物であることをコンパクトに示します。隠しファイル、非公開ファイル、または一時ファイルであることは意味しません。一般利用者向けの主導線とは分けますが、必要に応じて開発者向けページから取得できるようにします。

player：配布・再生用

player は、editor と同じ画像・音声に加えて、同時生成した変換済み台本を SB3 内へ組み込みます。浦島太郎では urashima.sb3 を使用します。アプリ起動後に表示されるタイトル画面をクリックすると、組み込み台本をランタイム変数 script へ設定し、ファイル選択を行わずに startStory を実行します。台本固有の画像・音声も同じ SB3 から解決し、アセット配信サーバから取得しません。

この保証は台本と物語固有アセットを対象とします。TMPose モデル、カメラ、その他の外部サービスなど、別途オンライン利用を前提とする機能まで完全にオフライン化するものではありません。オンライン依存は成果物 manifest と公開ページへ明記します。

TurboWarp Packager で HTML を生成する場合は、player だけを入力にします。editor を Web 版へ変換して、実行時に台本ファイル選択を要求する構成にはしません。サンプル一覧では Web 版と player を一般向けに案内し、editor は物語作成者向けの導線へ分離します。

生成と検証の契約

- generic は本体リポジトリで生成し、公開サンプルの台本・固有アセットを含めません。
- editor と player はサンプルリポジトリで、固定バージョンの本体ビルダー、同じビルド元台本、アセットロック、名前対応表から生成します。
- 生成設定と manifest へ profile を記録し、入力台本、アセット、ベース SB3、各成果物の SHA-256 を対応付けます。
- editor の外部台本と player へ組み込む台本は同じ変換結果を使用します。内容またはアセット参照がプロファイル間で分岐してはいけません。
- 同じ入力と固定依存から各プロファイルを再生成した場合、SB3、台本、manifest のハッシュが一致しなければなりません。
- player は、タイトルクリック後にファイル選択を開かず最初のシーンへ進み、台本固有アセットの HTTP 取得を行わないことを VM テストとブラウザテストで確認します。

このプロファイル契約は導入済みです。本体の台本組み込みとタイトルクリック開始は完了済みの [Issue #60](#)、浦島太郎の editor / player 生成は完了済みの [tmpose-kamishibai-samples Issue #2](#)、Packager Web 版は完了済みの同 [Issue #7](#) に実装履歴を記録しています。現在の公開物は、上記の生成と検証の契約を満たすものとしてビルド・検証します。

問題が見つかった場合、Web 版と player の公開を停止し、台本を外部から開く editor へ切り戻します。ビルダーの不具合では固定依存を直前の検証済みバージョンへ戻します。サンプル固有ファイルを本体リポジトリへコピーしてロールバックしてはいけません。

2. 内部仕様: skipMode と skipContext

skipMode はタイトル表示状態または未処理の進行要求を保持し、skipContext はキー入力を受け付けてよい実行文脈を示します。どちらも Temporary Variables 拡張のランタイム変数です。要求と実行文脈を分けることで、停止中の入力やアクション間の右矢印が、次の処理へ持ち越されることを防ぎます。

2.1 変数の表現

通常状態は、skipMode が**存在しない状態**で表します。通常状態用の文字列は使いません。廃止済みの none を設定または比較してはいけません。

変数	値	意味
skipMode	変数なし	未処理の要求なし
skipMode	title	タイトルを表示中
skipMode	pose	現在のポーズ1件を完了する要求
skipMode	action	現在のアクションを完了する要求
skipMode	scene	現在のシーンを完了する要求
skipContext	変数なし	タイトル、表紙、停止中など、シーン外
skipContext	scene	シーンの準備またはコマンド解析中
skipContext	betweenActions	アクション列の実行中だが、個別アクションの外側
skipContext	action	個別アクションの実行中
skipContext	pose	ポーズ待ちの実行中

通常状態へ戻すときは、skipMode を削除します。空文字列や通常状態用の値を代入しません。skipContext は処理の入口で設定し、表紙、タイトル、シーン終了時に削除します。

2.2 入力の受理

文脈	スペース	右矢印	下矢印
タイトル表示中	<code>title</code> を削除して表紙へ進む	無視	無視
ポーズ待ち中	<code>pose</code> を設定	<code>action</code> を設定	<code>scene</code> を設定
ポーズ以外のアクション実行中	無視	<code>action</code> を設定	<code>scene</code> を設定
アクション間	無視	無視	<code>scene</code> を設定
シーンの準備・解析中	無視	無視	<code>scene</code> を設定
シーン外・停止中	無視	無視	無視

入力ハンドラは、`skipMode` が存在しない場合だけ要求を設定します。したがって、最初に受理した要求が消費されるまで、後続入力による上書きや `pose` から `action` / `scene` への格上げは行いません。

2.3 要求の消費と伝播

`pose` は現在のポーズ処理だけが消費し、同じ `pose` アクション内の次のポーズへ進みます。`action` と `scene` はポーズ処理では削除せず、外側へ伝播します。`action` は現在のアクションが完了状態になった後にアクション列が削除し、次のアクションを実行します。`scene` はアクション列を抜け、シーン終了処理の後にシーン境界で削除します。

時間を伴う処理は、要求を受けると次の完了状態へ移してから外側へ戻ります。

処理	中断後の完了状態
<code>wait</code>	待機を終了
秒数付き <code>say</code> / <code>think</code>	吹き出しを消す
<code>moveTo</code>	指定した目的座標へ移動
<code>transition:fadeOut</code>	明るさを <code>-100</code> にする
<code>transition:fadeUp</code>	明るさを <code>0</code> にする
バックグラウンド実行中の <code>sequence</code>	最後の画像を適用して一回再生を終了
<code>sound</code>	<code>actionParam</code> が示す対象音声だけを停止

`sequence` は開始後すぐ次のアクションへ進むため、その後のアクション中に右矢印または下矢印が受理された場合も、実行中の一回再生を最終画像へ確定します。`loop` は継続演出なので、この確定処理の対象外です。

右矢印は現在の `sound` だけを停止します。`bgm` やほかの音声は、別のアクションを右矢印で完了しても停止しません。下矢印でシーンを終了する場合は、そのシーンで再生中の音声をすべて停止します。

緑の旗、停止、表紙開始、ストーリー開始、シーン開始・終了では、前の `skipMode` を残しません。表紙、タイトル、シーン終了時には `skipContext` も削除します。

2.4 実装上の不変条件

1. 通常状態は `skipMode` が存在しない状態だけで表す。
2. `skipMode` に設定する値は `title`、`pose`、`action`、`scene` だけにする。
3. `skipContext` に設定する値は `scene`、`betweenActions`、`action`、`pose` だけにする。
4. 入力受理時は `skipMode` の不在と `skipContext` の値を両方確認する。
5. `pose` だけをポーズ処理内で消費し、`action` と `scene` はそれぞれの外側の境界へ伝播する。
6. 完了状態を持つ時間処理は、その状態を適用してから要求を外側へ返す。
7. 表紙、タイトル、シーン境界で、消費済みの要求と実行文脈を残さない。

2.5 テスト方針

`test/skip-mode.test.mjs` は生成SB3のScratchブロックグラフを検査し、許可値、存在判定、要求の削除などの構造上の不変条件を確認します。ブロックIDは固定せず、オペコード、入力値、カスタムブロック名、親子関係を使います。

`test/turbowarp-vm.test.mjs` は `pretest` が `app/` から生成した `tmp/kamishibai.sb3` を、固定コミット `c4823421cb7c17d8d8a89878851ce1668c26a21f` のTurboWarp VMへ読み込みます。緑の旗とキー入力をVMへ送り、入力文脈、最初の要求の保持、ポーズからの伝播、シーン境界、待機、吹き出し、移動先、フェード最終値、画像列、対象音声の停止、BGMの継続を実行結果から検証します。Loadingについては、指定アセットの優先登録順、通常アセットだけを数える $0 / N$ から N / N までの進捗、複数画像の循環、完了後の非表示を検証します。

外部URLの機能拡張、カメラ、ネットワーク、音声出力は決定的なテストダブルへ置き換えます。テスト中に外部通信は行いません。VM内部APIへの依存は `test/helpers/turbowarp-vm.mjs` に隔離します。

通常のテストは次のコマンドで実行します。

```
pnpm test
```

別のSB3を構造検査するときは、対象を環境変数で指定できます。

```
KAMISHIBAI_SB3_PATH=/path/to/project.sb3 node --test test/skip-mode.test.mjs
```

2.6 wait のタイマー制御

`wait` アクションはTurboWarp標準のMore Timers拡張を使用します。同拡張のタイマーは `start/reset timer` で0に戻り、その後は経過秒数が増加します。そのため、待機中の継続条件は「タイマー値が指定秒数未満」でなければなりません。

実装はタイマーをリセットしてから、タイマー値が指定秒数未満であり、かつ `nextSceneLabel` と `skipMode` のどちらも存在しない間だけ短時間のyieldを繰り返します。ループを抜けた後はタイマーを削除します。指定秒数を設定した直後にリセットする実装や、`timer > 0` を継続条件にする実装は使用しません。

`test/wait-action.test.mjs` は、生成SB3のブロックグラフから、このカウント方向、終了値、シーン移動・スキップ用ガード、タイマー削除を検査します。

3. 関連ライブラリ

tmpose-kamishibaiの動作基盤として開発したもののうち、他のプロジェクトでも使えるものとして抽出し、オープンソースライセンス(MPL2.0)で公開しているライブラリを以下に列挙します。

3.1 Vite プラグイン

- vite-plugin-turbowarp-extension: A Vite plugin for building TypeScript projects as single-file TurboWarp extensions.
 - <https://github.com/kubohiroya/vite-plugin-turbowarp-extension>

3.2 TurboWarp 機能拡張開発用テンプレート

- turbowarp-extension-template: A reusable TypeScript template for developing, testing, building, and releasing TurboWarp extensions with Vite.
 - <https://github.com/kubohiroya/turbowarp-extension-template>

3.3 TurboWarp 機能拡張

- tmpose.js: A TurboWarp extension for camera-based pose recognition using Teachable Machine Pose models.
 - <https://github.com/kubohiroya/turbowarp-tmpose>
- text-lines.js: A TurboWarp extension for counting, reading, and splitting text by lines.
 - <https://github.com/kubohiroya/turbowarp-text-lines>

- `asset-manager.js`: An IndexedDB-backed image and audio asset manager for TurboWarp projects. It can also register costumes, stage backdrops, and sounds already stored in the current `.sb3` project.
 - <https://github.com/kubohiroya/turbowarp-asset-manager>
- `async-input.js`: A target-scoped asynchronous keyboard, pointer, and accumulated pose input extension for TurboWarp Temporary Variables.
 - <https://github.com/kubohiroya/turbowarp-async-input>
- `runtime-expression.js`: A safe JavaScript-like condition evaluator and conditional broadcast monitor for TurboWarp Temporary Variables runtime variables.
 - <https://github.com/kubohiroya/turbowarp-runtime-expression>

3.4 その他のライブラリ

- `rubygana.js`: A node.js library for converting Japanese text to kana.
 - <https://github.com/kubohiroya/rubygana>

4. 関連ドキュメント

- `01-user-guide.md`: 紙芝居アプリの操作方法と用途別成果物の使い分け
- `02-dsl-manual.md`: 紙芝居 DSL ファイルの作り方
- `03-command-reference.md`: コマンドとアクションの詳細仕様
- `history.md`: 紙芝居 DSL 2.0 から 3.1 への変更履歴